Pole-Placement Control of an Inverted Pendulum

OBJECTIVE.

Design a state-feedback controller that returns a perturbed pendulum to upright and maintains balance under random cart forces, using the nonlinear Simulink model from Project 1.

METHODOLOGY.

We retain the aligned Simulink plant, its frictionless cart, rigid massless rod, point pendulum mass, and pivot damping. The cart mass is 2 kg, pendulum mass is 0.5 kg, rod length is 1 m, pivot damping is 0.01 N m s/rad, and gravity is 9.81 m/s². Here x is cart position and θ is angle from upright. Near upright, $\sin(\theta) \approx \theta$, $\cos(\theta) \approx 1$, and the centrifugal term $\sin(\theta)\dot{\theta}^2$ has no first-order contribution. With $z = \left(x, \dot{x}, \theta, \dot{\theta}\right)^T$, the linearized model is

$$\dot{z} = Az + BF, \quad A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -2.4525 & 0.005 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 12.2625 & -0.025 \end{pmatrix}, \quad B = \begin{pmatrix} 0 \\ 0.5 \\ 0 \\ -0.5 \end{pmatrix}.$$

The controllability matrix $\mathcal{C} = [B, AB, A^2B, A^3B]$ has rank four. Therefore, all four closed-loop poles can be assigned through state feedback.

We choose poles $p = (-1 + i, -1 - i, -3, -4)$ s⁻¹. The dominant pair has damping ratio $\frac{1}{\sqrt{2}} = 0.707$ and an exponential envelope e^{-t}, giving an approximate 2% settling envelope of 4 s. The two remaining poles decay three and four times faster. These choices give a moderately damped response without demanding arbitrarily fast actuation. They describe the local, unlimited linear model; saturation and nonlinear motion are checked by simulation.

The desired characteristic polynomial is $\varphi(s) = (s^2 + 2s + 2)(s + 3)(s + 4)$. We calculate the gain directly using Ackermann's formula,

$$K = (0 \;\; 0 \;\; 0 \;\; 1)\mathcal{C}^{-1}\varphi(A), \quad u = -Kz.$$

This gives $K = (-4.893, -7.7572, -85.5731, -25.7072)$. MATLAB verifies that the eigenvalues of $A - BK$ match the selected poles and independently checks the gain with `place` [1]. We apply this gain to the existing Simulink diagram. The actuator clips u to ± 10 N, and the external disturbance d is added afterward, giving $F = \mathrm{sat}(-Kz) + d$.

We run 30 s nonlinear tests from 5, 10, 20, and 30° releases, followed by tests from upright with random forces bounded by ± 2.5, 5, 7.5, and 10 N. Independent uniform draws are held for 0.1 s; seed 79202 supplies the same sequence at each bound. The variable-step solver uses a maximum step of 0.01 s. Analysis uses 0.02 s samples. A run loses balance when $|\theta| \geq 90°$; failed traces stop there. Appendix A shows the retained diagrams, and Appendix B gives essential implementation code.

RESULTS.

For the 5° release, the nonlinear model settles within $\pm 1°$ and ± 0.05 m after 2.76 s and stays there through 30 s. Peak cart travel is 0.28 m, and peak applied force is 7.47 N. Figure 1 shows how increasing the release angle changes recovery and actuator use. Saturation limits the corrective force even though the unlimited linear poles remain stable.

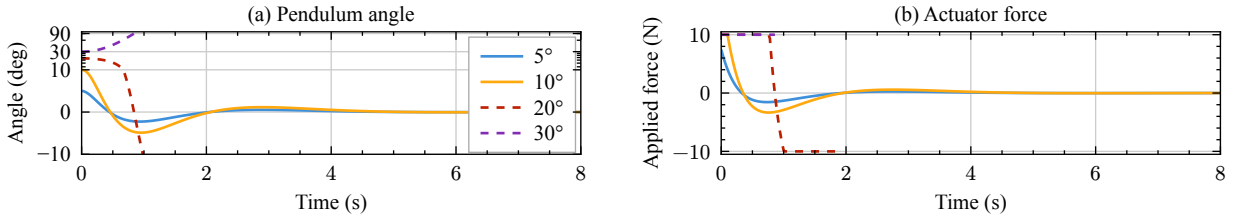


Figure 1: Nonlinear release tests, showing the first 8 s. Dashed traces lose balance and end at the first sampled 90° crossing. The force is clipped to ±10 N. The angle axis is linear within ±10° and logarithmic beyond.

The 5 and 10° releases recover. The 20 and 30° releases lose balance. The 10° release recovers after 3.32 s, with 0.11 s of saturation. The 20° case reaches 90° at 1.92 s. Thus, the tested transition from recovery to failure lies between 10° and 20° for this gain and actuator limit.

The random-force tests in Figure 2 evaluate a different demand. Instead of removing one initial displacement, the controller must continually oppose an external force. At the 2.5 N bound, angle RMS is 0.77° and peak angle is 2.41° over 30 s. Table 1 summarizes whether each input maintains balance.

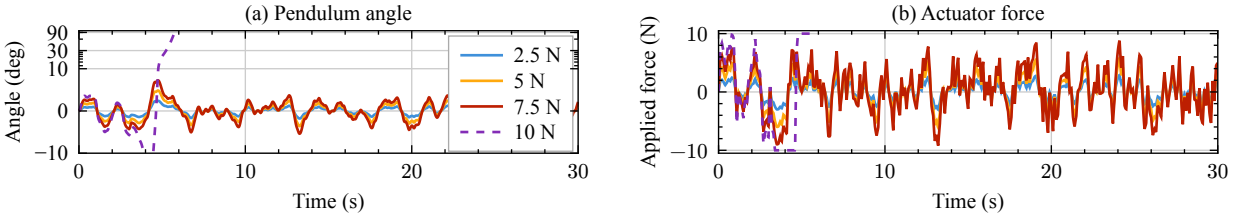


Figure 2: Nonlinear random-force tests over 30 s. Bounds label the external disturbance; panel (b) shows the separate actuator force. Dashed traces lose balance. The angle axis is linear within ±10° and logarithmic beyond.

Table 1: Random-force outcomes. RMS uses the full 30 s for balanced runs; failed-run travel stops at the first sampled 90° crossing.

Force bound (N)	Balanced	Angle RMS (deg)	Peak cart travel (m)
2.5	Yes	0.77	0.21
5	Yes	1.53	0.42
7.5	Yes	2.3	0.63
10	No	n/a	3.32

DISCUSSION.

The controller meets the near-upright recovery objective with a finite actuator. The 10 N random-force case reaches 90° at 5.76 s, after 2.19 s of actuator saturation. The lower three bounds remain balanced for all 30 s. Cart travel also matters, since the model has no physical end stops. Random-force outcomes refer to this recorded sequence; the bounds define amplitude, not a guaranteed disturbance-rejection limit. The report therefore separates local pole stability from the nonlinear, force-limited performance observed in these tests.

Appendix A. Simulink implementation.

The existing Project 2 layout is retained. The gain from Appendix B is embedded in the feedback block; the actuator and disturbance connections are unchanged.

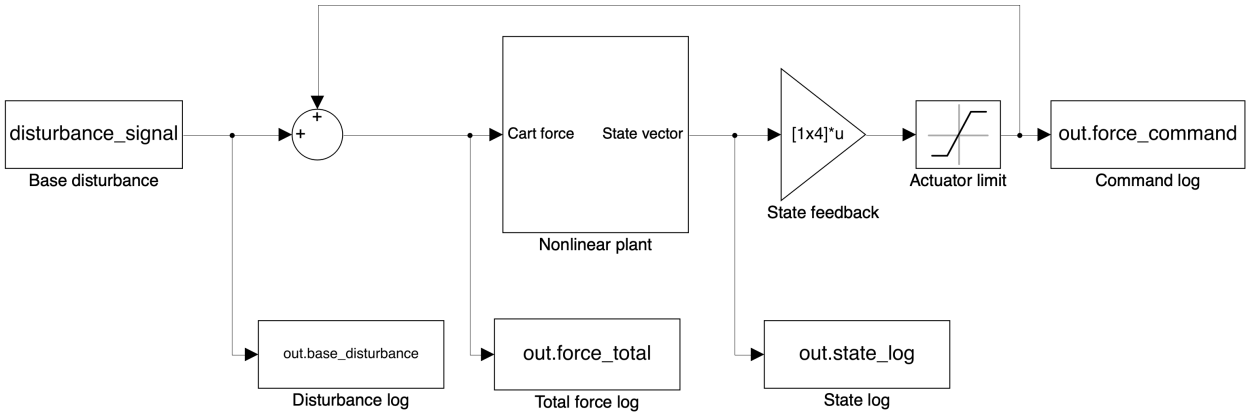


Figure 3: Retained feedback loop. The external disturbance is added after the actuator limit.

The nonlinear plant integrates angular and cart accelerations. The State order block displays $\left(\theta, \dot{\theta}, \dot{x}, x\right)$ and outputs $\left(x, \dot{x}, \theta, \dot{\theta}\right)$ for feedback.

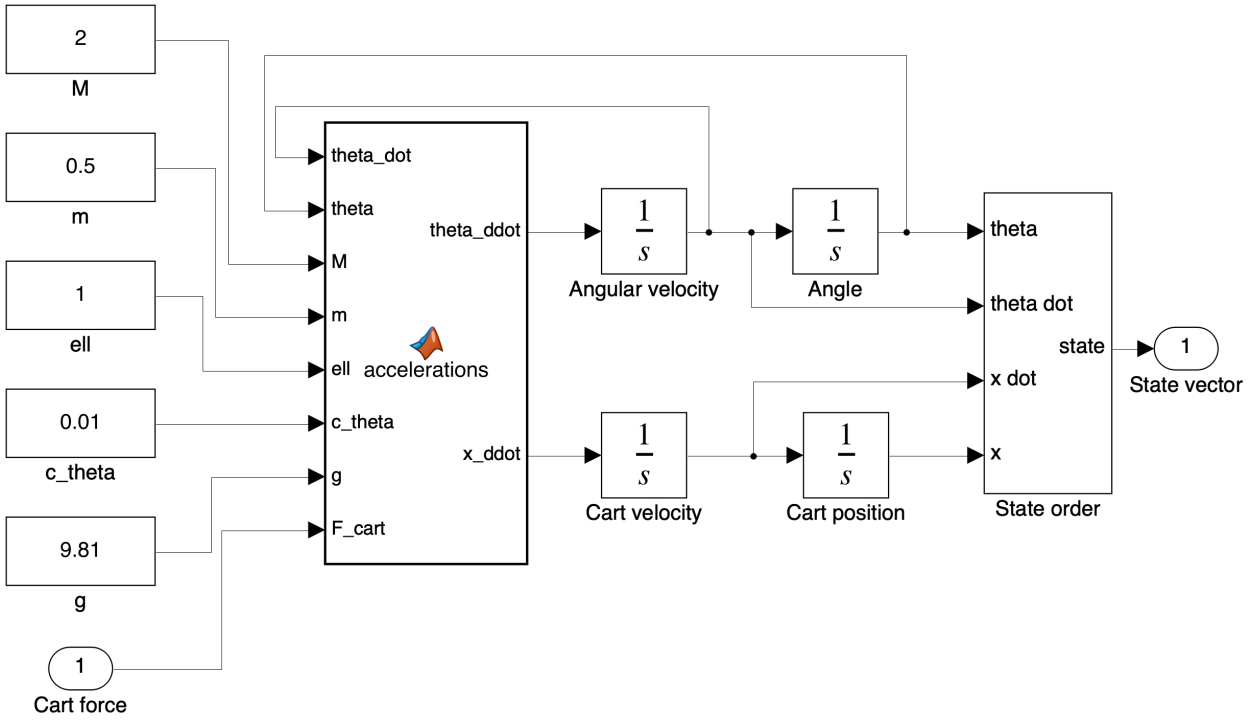


Figure 4: Retained nonlinear plant and four integrators from Project 1.

Appendix B. Essential MATLAB code.

The first function calculates the pole-placement gain for the state order used by the model. The nonlinear acceleration function below is embedded in the Simulink plant. Experiment automation remains in the accompanying `run_pole_revision.m` file.

```
function [K, A, B, poles] = design_pole_controller()
% State order is [x; x_dot; theta; theta_dot], with theta in radians.
M = 2; m = 0.5; ell = 1; c = 0.01; g = 9.81;
A = [0 1 0 0; 0 0 -m*g/M c/(M*ell); 0 0 0 1; ...
     0 0 (M+m)*g/(M*ell) -c*(M+m)/(M*m*ell^2)];
B = [0; 1/M; 0; -1/(M*ell)];
% Dominant pair gives damping 0.707 and an approximately 4 s envelope.
poles = [-1+1i -1-1i -3 -4];
a = poly(poles);
C = [B A*B A^2*B A^3*B];
% Ackermann's formula assigns the desired characteristic polynomial.
phi = A^4 + a(2)*A^3 + a(3)*A^2 + a(4)*A + a(5)*eye(4);
K = ([0 0 0 1]/C)*phi;
end
```

```
function [theta_ddot, x_ddot] = accelerations(theta_dot, theta, M, m, ell, ...
    c_theta, g, F_cart)
% Nonlinear cart-pendulum equations with an applied horizontal cart force.
% Positive theta is the upright perturbation used in Mini Project 1.

s = sin(theta);
c = cos(theta);
mass_matrix = [M + m, m * ell * c; m * ell * c, m * ell^2];
forcing = [F_cart + m * ell * s * theta_dot^2; ...
           m * g * ell * s - c_theta * theta_dot];
acceleration = mass_matrix \ forcing;

x_ddot = acceleration(1);
theta_ddot = acceleration(2);
end
```

To run the two required tests directly, use a 5° release without disturbance, then start upright with the held 2.5 N disturbance. The runner assigns the input data to the model workspace to make the updated model independent of external scripts.

```
model = 'inverted_pendulum_controlled';
load_system(model);
[K,~,~,~] = design_pole_controller();
set_param([model '/State feedback'],'Gain',mat2str(-K,17));
w = get_param(model,'ModelWorkspace');
assignin(w,'disturbance_signal',[0 0;30.1 0]);
set_param([model '/Nonlinear plant/Angle'],'InitialCondition','5*pi/180');
release = sim(model);
rng(79202,'twister'); t = (0:0.1:30.1)';
assignin(w,'disturbance_signal',[t 2.5*(2*rand(size(t))-1)]);
set_param([model '/Nonlinear plant/Angle'],'InitialCondition','0');
random_force = sim(model);
```

References.

[1] MathWorks, “place. Pole placement design.” Accessed: Oct. 16, 2026. [Online]. Available: https://www.mathworks.com/help/control/ref/place.html